

CHARUSAT UNIVERSITY

DEPARTMENT OF INFORMATION TECHNOLOGY

INFOMINE 

Offline Intelligent Document RAG System

Submitted By:

Ayush Desai

ID: 24IT012

Harsh Thesiya

ID: 24IT099

Course: SGP Project (2025-26)

Faculty: Information Technology Department

Submission Date: May 1, 2026

1. Abstract

INFOMINE is a robust, intelligent, and 100% offline document-based chatbot built with Streamlit and Python. It ensures complete data privacy by processing PDF documents locally using open-source models. The system integrates Retrieval-Augmented Generation (RAG) to provide accurate answers, concise summaries, and automated quiz generation. By utilizing `flan-t5` and `FAISS`, INFOMINE offers a powerful AI experience without the need for internet connectivity or expensive API keys.

2. Introduction

2.1 Project Overview

The INFOMINE project addresses the growing need for secure, private document analysis tools. In an era where data leaks are frequent, INFOMINE provides a "local-first" solution for students, researchers, and professionals who need to interact with sensitive documents safely.

2.2 Objectives

- Develop a user-friendly interface for uploading and managing multiple PDF documents.
- Implement an offline vector store for efficient document indexing.
- Enable natural language interaction with documents using local Large Language Models (LLMs).
- Automate the creation of self-assessment quizzes from document content.

3. System Requirements

3.1 Functional Requirements

- **Offline Processing:** All AI computations must occur on the local CPU/GPU.

- **RAG Chat:** Context-aware questioning based on the uploaded PDF contents.
- **Summarization:** Generate structured summaries using Map-Reduce techniques.
- **Quiz Generation:** Create MCQs and True/False questions for user testing.

3.2 Tech Stack

- **Language:** Python 3.9+
- **Frontend:** HTML, CSS, JS
- **Models:** google/flan-t5-base (LLM), all-MiniLM-L6-v2 (Embeddings)
- **Vector Store:** FAISS (Facebook AI Similarity Search)
- **Orchestration:** LangChain

4. System Design

The system is designed around a modular RAG pipeline. When a user uploads a PDF, the text is extracted using `PyPDF2`, split into chunks, and converted into mathematical vectors (embeddings). These vectors are stored in a local FAISS index.

4.1 Architecture Flow

1. **Data Ingestion:** PDF loading and text splitting.
2. **Indexing:** Storing vector representations in the FAISS database.
3. **Retrieval:** Finding relevant text chunks based on user queries.
4. **Generation:** Using `flan-t5` to generate answers from the retrieved context.

5. Implementation

The implementation focuses on memory efficiency, ensuring the model fits within 4-8GB of RAM. The default `flan-t5-base` model was chosen for its balance between performance and resource consumption.

5.1 Core Logic Snippet

```
# Loading the Offline LLM via LangChain
```

```

from langchain.llms import HuggingFacePipeline
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, pipeline

def load_local_llm():
    model_id = "google/flan-t5-base"
    tokenizer = AutoTokenizer.from_pretrained(model_id)
    model = AutoModelForSeq2SeqLM.from_pretrained(model_id)
    pipe = pipeline("text2text-generation", model=model,
tokenizer=tokenizer)
    return HuggingFacePipeline(pipeline=pipe)

```

6. Testing

The system underwent rigorous testing to ensure offline stability and accuracy.

Test Component	Method	Result
PDF Extraction	Multi-page PDF test	Successfully parsed text from complex layouts.
Offline Verification	Network Disconnect	Model generated responses with zero internet access.
Summarization	Map-Reduce check	Accurately summarized a 50-page document.

7. Team Contribution

Member Name	Role	Primary Contributions
Ayush Desai	UI & Feature Lead	HTML, CSS, JS dashboard development, Quiz generation logic, and PDF preprocessing.

Harsh Thesiya	Backend	Model loading logic, FAISS vector store integration, and RAG pipeline optimization.
--------------------------	---------	---

8. Conclusion

INFOMINE demonstrates that high-quality document intelligence does not require cloud connectivity. By integrating local LLMs with RAG architecture, we have built a privacy-preserving tool suitable for academic and professional use. Future work will focus on integrating larger quantized models like Llama 3 for enhanced reasoning capabilities.